Financial forecasting — project plan

Reference material: `references/` (index in `references/README.md`).

Decision proposals

To be decided by the team.

- Scope: level 1 complete (full chain, persistence + mean-change baselines, historical replay, trading calendar), with several securities held in the SQL database from the start, plus the level 3 parts the course requires (prediction logging, drift checks, triggered retraining, keep-the-better-model rule). Level 2 items only after the chain runs end to end; cheapest first: simple sliding-window backtest. Alternatives:
  - Level 1 + course-required level 3 parts, level 2 later (proposed): covers every course checklist item with the least model work; the evaluation stays simple until the chain runs.
  - Level 1 only: least work, but misses parts the phase 3 checklist requires (prediction logging, drift checks, triggered retraining) — not viable for the course.
  - Level 1 + full level 2 from the start (several dozen securities, one global model vs. one per security, sliding-window backtest, prediction intervals): stronger evaluation, but it is model work, which is not graded, and it delays the end-to-end chain.
  - Full level 3 (telling volatile markets apart from model degradation, monitoring per security or sector): goes beyond what the course asks; only if time is left.
- First model: at most 1–2 days of model work (coach), then on to the MLOps parts. Proposal: Ridge regression (scikit-learn) predicting the next-day change, always scored next to the two required baselines (persistence: predicted change 0; historical mean change). Features from the price history only: changes of the last 1–5 days, 5- and 20-day volatility, volume change. Metrics: MAE, skill against persistence, directional accuracy, on days after the training period. Candidates with pros and cons: `decisions/regressors.md`.
- Build budget: ~2 weeks of focused work; chain first (one rough version of every stage end to end), polish after. Alternatives:
  - Chain first, polish after (proposed): a working end-to-end system exists early, so integration problems show up early and there is always something to demo; early versions are rough.
  - Stage by stage, each finished before the next starts: every part is polished when done, but the chain is complete only at the end, integration problems appear late, and a delay leaves an incomplete system at the defense.
  - Follow the course phase deadlines only, without an own budget: matches the coach's checkpoints, but phase 3 (monitoring, retraining, app, docs) gets the least time before the defense.
- Code review: GitHub pull requests. Every change on its own branch and merged through a pull request from the first day; during the phase 1 prototype the author merges their own pull request (others may comment); from phase 2 (after Oct 12) a pull request needs one approval before it is merged. Reviewer: the next person in the ring (`docs/team.md`). Alternatives:
  - Pull requests from day one, approval required from phase 2 (proposed): three people can build in parallel without breaking each other's work, and the repo shows the Git workflow and reviews the course grades; the prototype is not slowed down by waiting for reviews.
  - Approval required from day one: the strongest review record, but every change waits for a second person during the four days to Oct 12.
  - Push straight to `main` during the prototype, pull requests from phase 2: fastest for a single person, but three people on `main` collide with half-finished work, and phase 1 leaves no record of collaboration.
  - Pair programming: code is reviewed while it is written; needs two people at the same time and leaves no review record in the repo.
- API access (phase 2, optional in the course): HTTP basic authentication against a `users` table in the project database (passwords stored as bcrypt hashes, see `decisions/password-hashing.md`; role `user` or `admin`). Users call `predict/`; admins also call `training/`, the model metrics and a few user-management endpoints (add, list, remove). The demo app logs in as a user. Builds on the course's API security exercise. Alternatives:
  - Basic authentication with a users table (proposed): few endpoints; users are managed without a code change; roles separate the user side from the admin side. The password travels with every request, so it needs HTTPS beyond a local machine.
  - One API key per client, read from `.env`: simplest; no users or roles, and a new key needs a restart.
  - Login with tokens (JWT, OAuth2 password flow): the password is sent once and the token expires; more code (sign-up, login, token checks) for little gain in a local demo.
  - No authentication: least work; misses the optional API security item, and anyone can start a training.
- Test strategy: requirements first. Each requirement names how it is checked: Test, Demonstration or Inspection (`requirements.md`, "How requirements are checked"). For every requirement checked by Test, black-box acceptance tests are written before the code (TDD); the share of passing acceptance tests is the progress indicator (`make acceptance`). Below them: unit tests for the pure functions (features, target, no look-ahead, baselines, metrics, trading calendar), API tests with swapped-in parts, and integration tests against a throwaway database with a small fixed price set. Each pytest-xdist worker gets its own test database; tests never use the network, a cloud service or real e-mail. No model-quality thresholds: tests check that the comparison with the baselines is computed, not that the model wins. Acceptance tests of phase 1 are in `tests/acceptance/`; those of phases 2 and 3 are written when each phase starts. Alternatives:
  - Acceptance tests first, unit tests with the code (proposed): progress is measured against the requirements, and red tests show what is missing; the interfaces have to be agreed early.
  - Unit tests only, written with the code: quick to start; nothing shows whether a requirement is met.
  - Manual checks and a demo script: no test code; nothing is repeatable, and the course's control points include test coverage.
- Requirements tool: StrictDoc; top level only (~15–25 requirements), detail grows with the code. Draft in `docs/requirements.sdoc`. After every change to it, run `make requirements` to regenerate `docs/requirements.md` for reading. Alternatives:
  - StrictDoc (proposed): each requirement has an ID that code and tests can refer to; checked by the tool, exported to readable Markdown; a text format the team has to learn.
  - Plain Markdown list in the repo: no tool, readable on GitHub at once; IDs and links are kept by hand, nothing checks them.
  - GitHub Issues or Projects: doubles as the task board (assignment, links to pull requests); requirements mix with tasks and are not versioned with the code.

Roadmap — proposal

To be agreed by the team. Team setup and areas: `docs/team.md`.

Target application, as the jury sees it at the defense: a Streamlit app that calls the model API.

- Pick a security; see its next-day forecast with the model version, next to both baselines; the "not investment advice" notice always visible.
- A history chart of past forecasts against the actual outcomes.
- A status panel: latest stored quote, latest training run, current production model, latest drift check.

Live demo: replay advances the simulated calendar by a few trading days and the chain runs on its own — new quotes are loaded, predictions logged, drift checked, the model retrained and promoted if it is better — visible in the app, MLflow and Grafana.

Period	Milestone	Content
until Mon Oct 12 (coach meeting 12:00)	Foundations: a clone plus `make` gives a database with history, a trained baseline and an API answering `predict/`	"Phase 1 plan" below; roadmap and team setup agreed
Oct 13–22 (coach meeting)	Chain runs end to end: one replayed day goes from new quotes through training and promotion to the API serving the new model	MLflow tracking and registry, "better model" rule, Docker Compose services (database, API, MLflow, Airflow), Airflow DAGs for ingestion and training, replay and trading calendar, DVC snapshots with hashes in MLflow
Oct 23–Nov 6	Closed loop and monitoring	prediction log and outcome collection, Evidently drift DAGs (training and prediction side), Prometheus and Grafana with alerts and the retraining webhook, first Streamlit version
Nov 7–11	Feature freeze Nov 11	tests, README and API docs, check from a fresh clone, demo script
Thu Nov 12	Mock defense call	full demo as at the defense
Nov 13–16	Polish and rehearse	fixes from the mock defense, presentation (slides or in Streamlit), 1–2 rehearsals with equal speaking time

Level 2 items (more securities, backtesting) start only if the chain is complete by Nov 6.

Phase 1 plan — foundations by Mon Oct 12

Proposal for the team. Coach meeting Mon Oct 12, 12:00 (invite sent; a time that does not suit is moved on request in Slack).

Goal: a teammate runs `git clone` and a handful of `make` commands and gets a running database with price history, a trained baseline model, and an API answering `predict/`.

#	Item	Done when	Depends on
0	Team meeting (before Oct 12)	roadmap draft and organisational setup agreed (see Open); database, data source and securities decided; objectives and key metrics (draft in `docs/requirements.sdoc`) confirmed	—
1	Repo ready and shared	team has write access; repo structure agreed (see Open); template README replaced; `.gitignore` and `.env.example` in place; stub CI passes (one trivial test) or is disabled until tests exist	0
2	Reproducible environment	`pyproject.toml`, `uv.lock`, `.python-version`; Makefile targets `setup`, `db`, `load`, `train`, `predict`, `api`, `test`; works from a fresh clone	1
3	PostgreSQL	Compose file with only the database (exact image tag, data volume, credentials from `.env`); table `prices` keyed by (security, day)	2
4	One-time load script	loads the history of the agreed securities; a re-run creates no duplicates; API key read from `.env`	3, data source
5	Baseline training pipeline	a training function reads prices from the database, builds the target (next-day change), trains, compares with the persistence and mean-change baselines, saves the model to a gitignored folder; a prediction function returns the next-day forecast for one security; one file or two (see Open)	3 (4 for real data)
6	Initial API endpoints	FastAPI with `training/` (runs training) and `predict/` (security in; forecast and model version out)	5
7	Minimum tests and docs	unit tests (pytest, run in parallel with pytest-xdist) for feature building and both endpoints; README section "Setup from a fresh clone"	2–6

Repo sharing (item 1): every member works with their own GitHub account; access is granted to the accounts, never shared. Each member brings their GitHub username to the meeting.

- Team repo: the course repo `DataScientest-Studio/sep26_bmlops_int_financial_pred`. Members get access through the coach's request for their GitHub usernames; one member already has admin rights and can invite the others.
- "Changes only through reviewed pull requests" is enforced by a branch protection rule on `main`; on a free plan only for public repos, otherwise by team discipline.

Interfaces to agree at the meeting, so items 3–6 can be built in parallel:

- Table: `prices(symbol, day, open, high, low, close, volume)`, primary key (symbol, day).
- Functions: `train() → model_version`, `predict(symbol) → forecast`.
- API: request and response of `training/` and `predict/`.

Split: by area (data, model, service), see `docs/team.md`.

In Phase 1 the model is saved as a plain file; MLflow follows in Phase 2 (see "Roadmap — proposal").

Open

- Roadmap: proposal in "Roadmap — proposal", to be agreed by the team before building (coach). Input still to review: the coach's previous group repos (`references/README.md`, past learner projects).
- Organisational setup: proposal in `docs/team.md` (areas and owners, meetings, communication, work tracking, reviews), to be agreed by the team.
- Repo structure: the course repo ships a cookiecutter scaffold (from the "movie_recommandation" subject: `data/`, `notebooks/`, `reports/`, `src/` split into data, features, models, visualization; `requirements.txt`); its README says "feel free to make some modifications". May we replace the structure with our own (for example, no notebooks), or is it meant to be adapted file by file? (Ask the coach.)
- Local operation (REQ-CON-002): the coach requires the whole project to run locally. Every member's pipeline needs an up-to-date database during development. Options: each member runs their own database and loads from the data source; a shared development database on a team member's NAS, reached over Tailscale; a small rented cloud server (for example, AWS). Is a shared database during development acceptable? (Ask the coach.)
- Database: PostgreSQL or NoSQL. Comparison and proposal in `decisions/database.md`.
- Data source: may the team use a paid EODHD plan during pipeline development (full history load, testing), with the free tier (20 calls per day, 1 year of history) serving the daily update of up to ~20 securities once the pipeline runs? To check: whether the paid plan's terms allow sharing its data with the team, and that the history loaded with it is kept (database dump or DVC snapshot), since the free tier cannot fetch it again. Experience from a working private system: Stooq blocks automated downloads, and Yahoo (yfinance) rate-limits repeated requests (HTTP 429); EODHD worked reliably.
- Close prices: the `prices` table holds one close price, so a stock split appears as a large false change. Store the raw close, the adjusted close (corrected for splits and dividends), or both? Decide before the load script is written: changing the table means rebuilding the database.
- Objectives and key metrics: draft in `docs/requirements.sdoc`, to be confirmed by the team; main metric and the "better model" rule still to be defined.
- Demo app: planned with Streamlit, which the coach lists as a deliverable. What is the reason for prescribing it? (Ask the coach.)
- Training and prediction code: one file or two (the coach accepts either). Proposal: separate modules for training, prediction and evaluation (`src/forecasting/`), as in the course scaffold and the past group projects.
- Defense date: the coach says Nov 17 or 18, the SEP26 sheet says Monday or Tuesday (Nov 16 or 17); the sheet also lists a mock defense call on Nov 12. (Ask the coach.)
- `requirements.txt`: the course scaffold ships one (the classic pip package list) and its stub CI may install from it. The project environment is managed by uv (`pyproject.toml`, `uv.lock`). Drop `requirements.txt`, or keep it as a file generated from `uv.lock` (`uv export --format requirements-txt`) for anyone using plain pip? Generated, it must be regenerated after every package change.
- Development process: how much is expected — a GitHub Actions workflow (the course lists one as optional: lint, tests, image build), ruff for linting and formatting, scheduled (nightly) test runs? (Ask the coach.) Code review is a team decision, see "Decision proposals".